Bus Architectures:

02: AXI4 - Lite

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Announcements

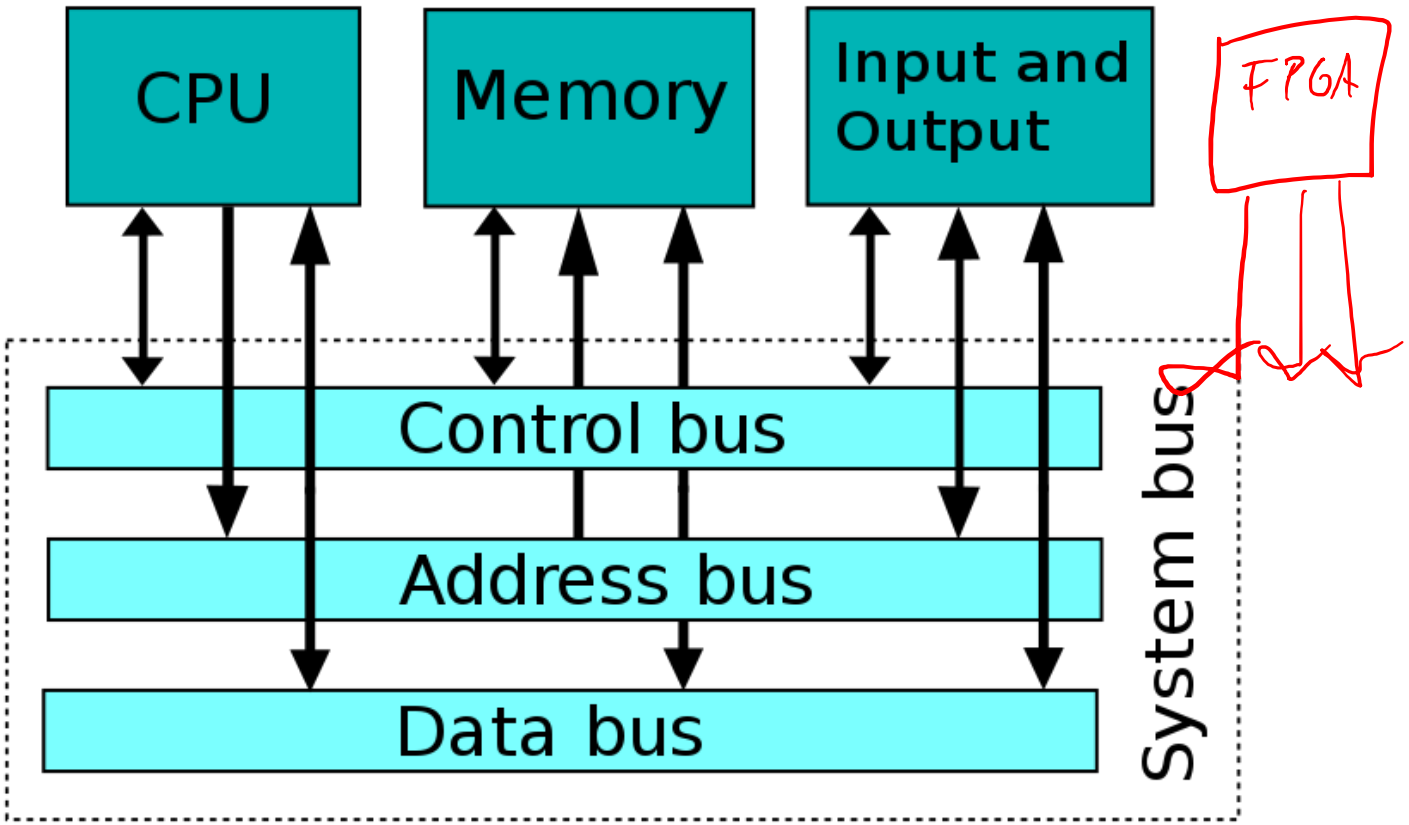
uint32_t
uint64_t
~~uint128_t~~

- engr315.github.io

- P2 Updates should be almost done.

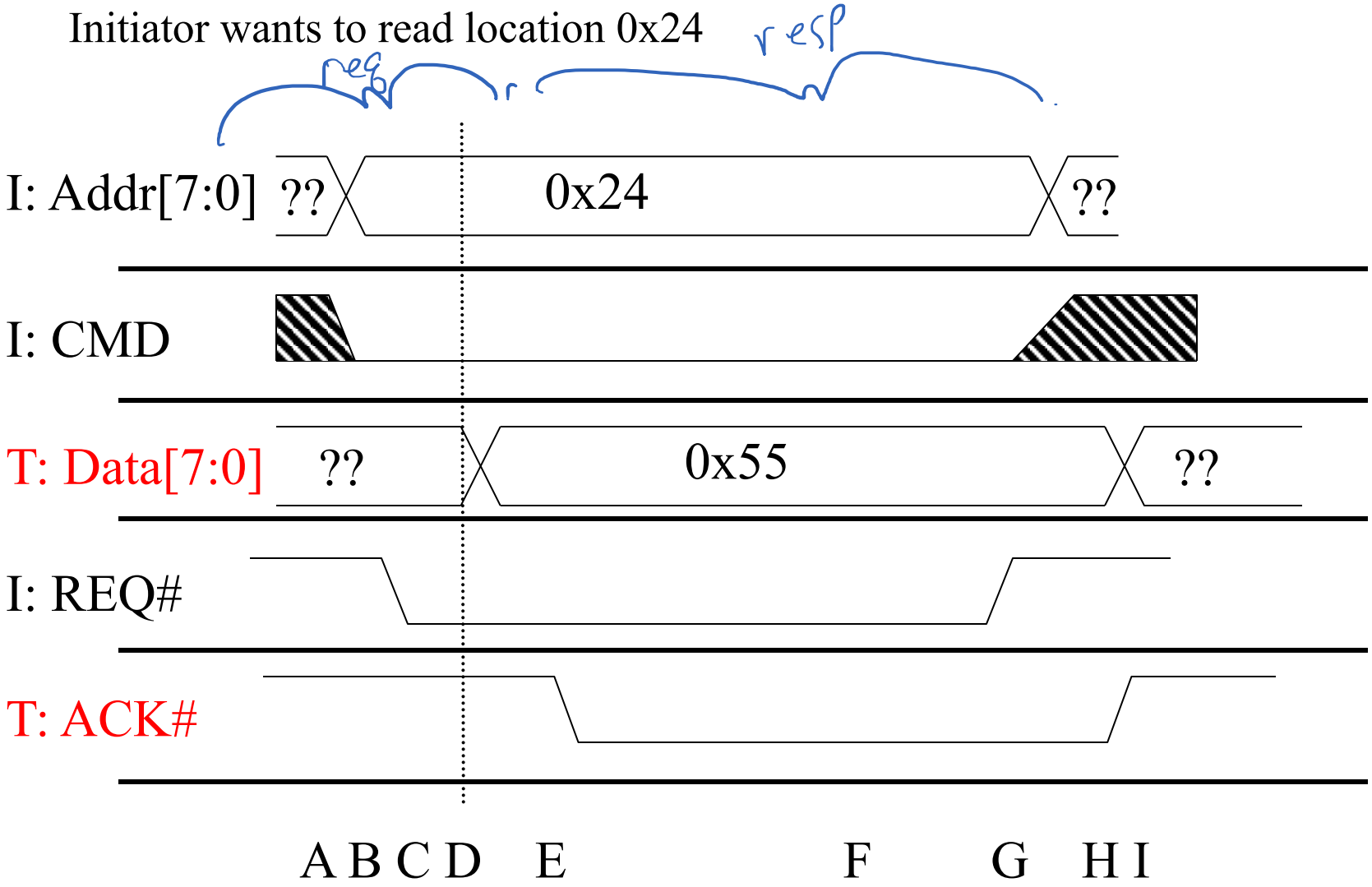
Ethan: office hours
5:30 Wed – 8:30 Wed

The System Bus



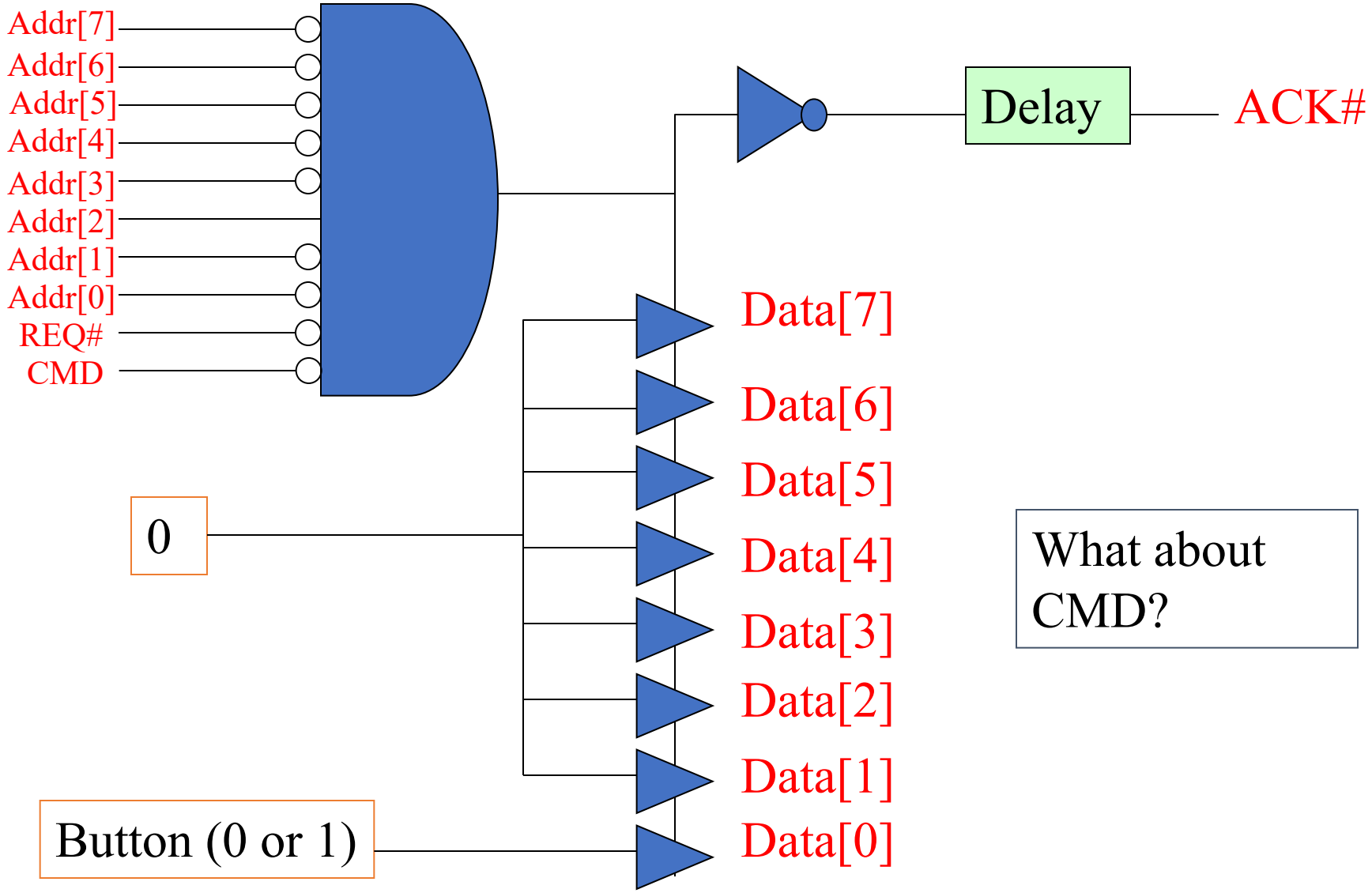
Read transaction

Initiator wants to read location 0x24



The push-button

(if Addr=0x04 write 0 or 1 depending on button)



ARM AXI Bus

- “Advanced eXtensible Interface” Bus Version 4, “AXI4”

ARM AXI Bus

- "Advanced eXtensible Interface" Bus Version 4, "AXI4"

e.g. V3

- Three Variants
  - AXI4: Fast but complicated; Memory-mapped

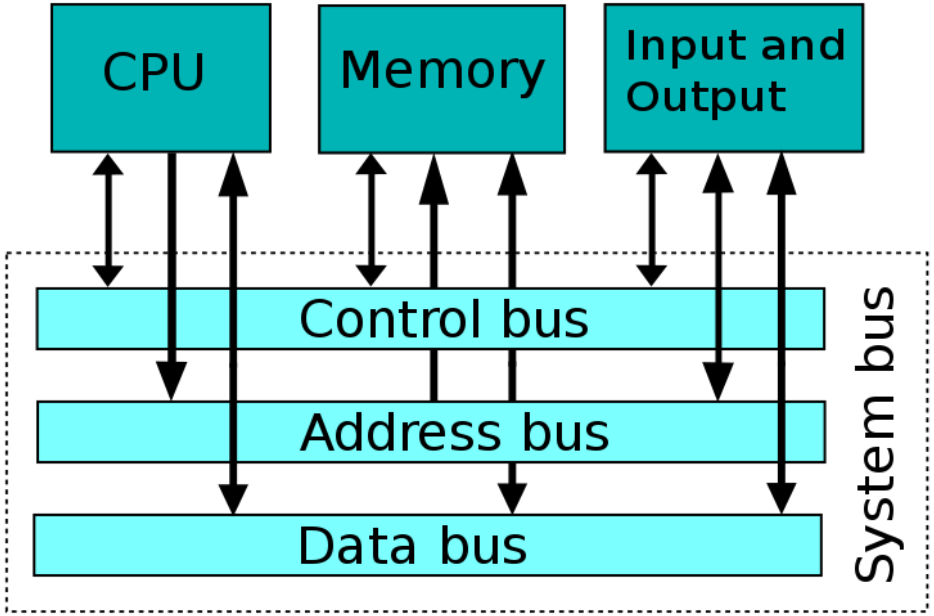
AHB

  - AXI4 Lite: Slow but simple; Memory-mapped

APB

  - AXI4 Stream: Fast and simple; Not memory-mapped

Why AXI4 Lite?

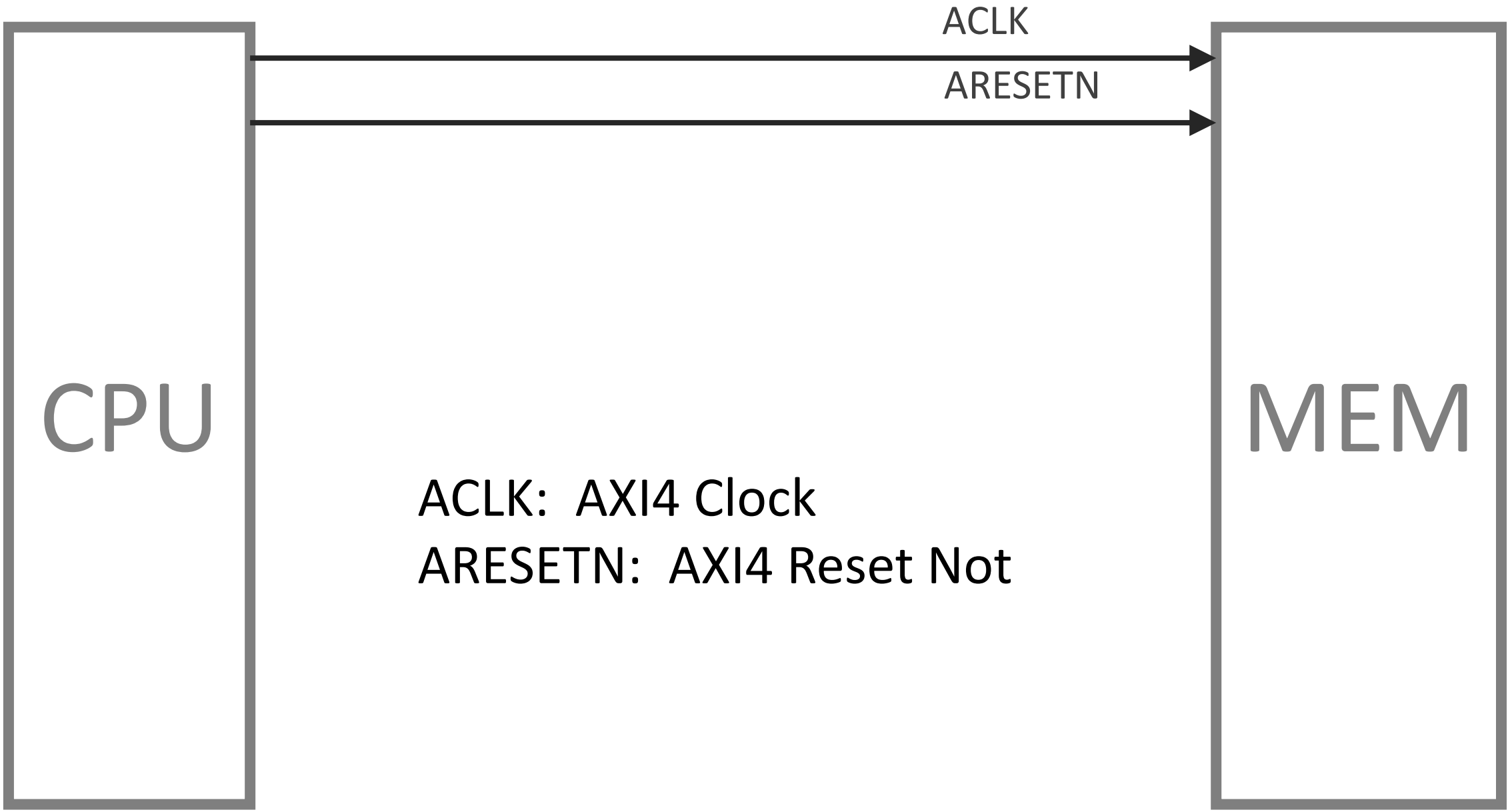


Xilinx AXI Reference Guide:

"AXI4-Lite is a light-weight, single transaction memory mapped interface. It has a small logic footprint and is a simple interface to work with both in design and usage. "

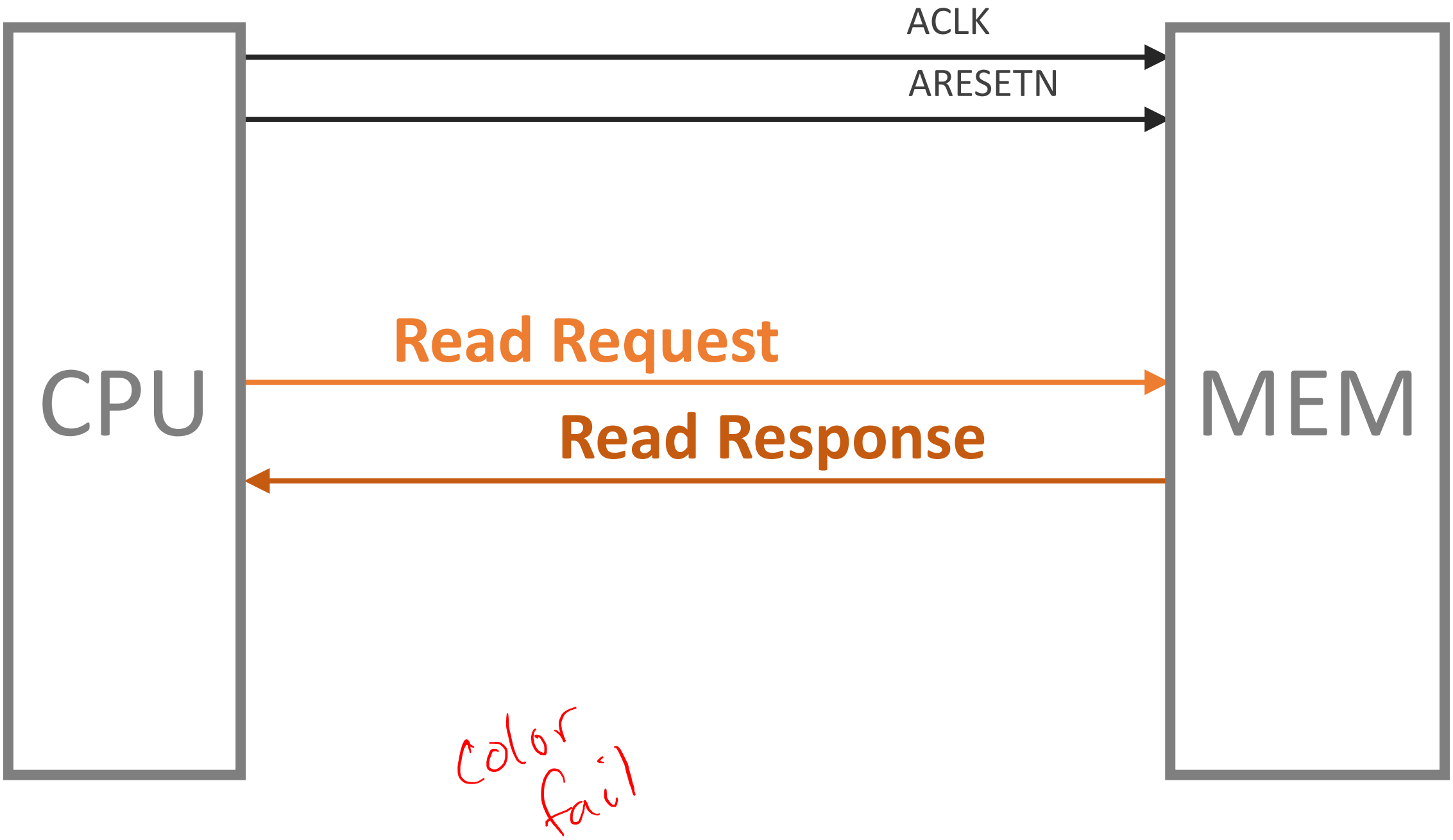
CPU

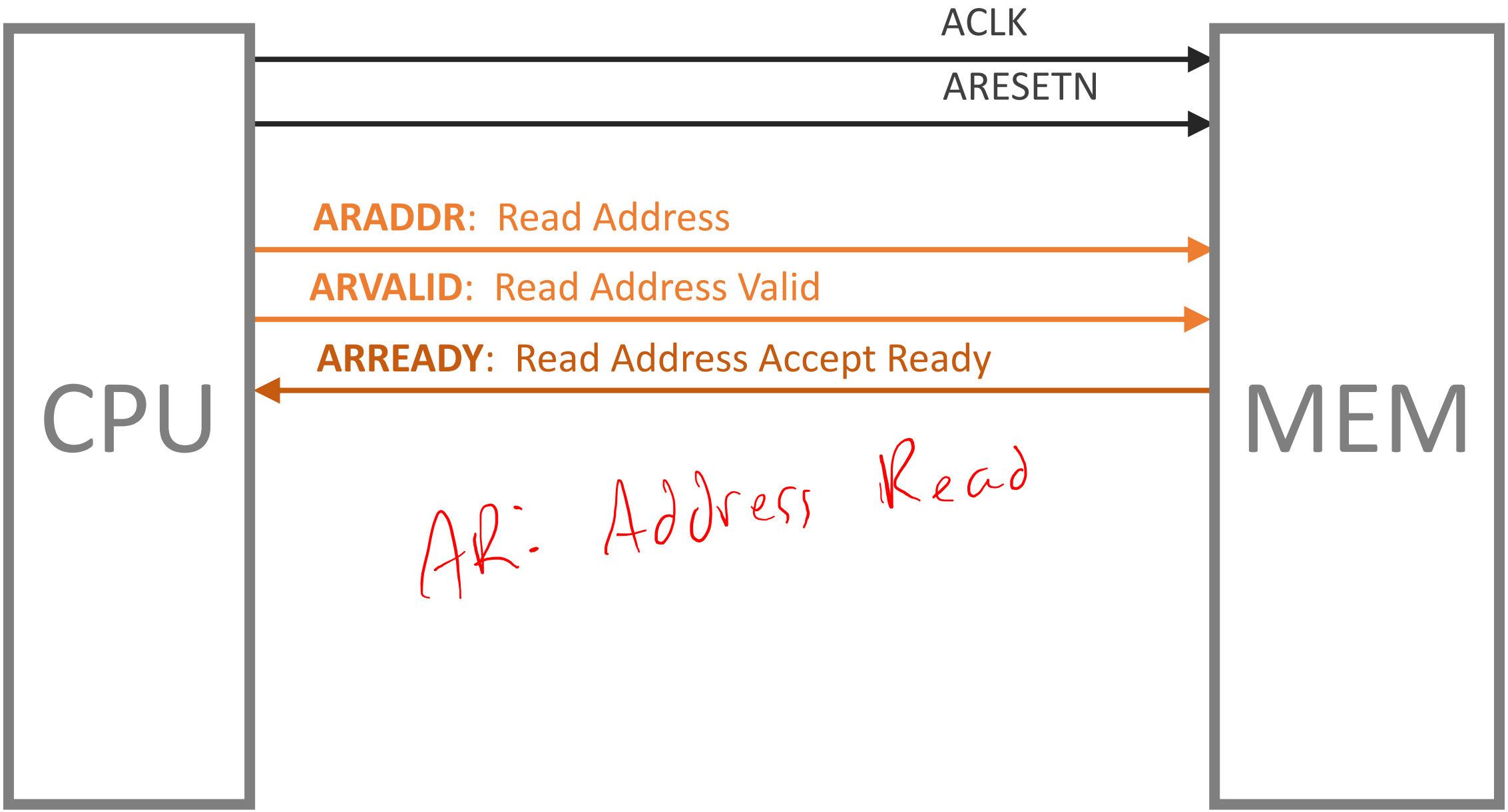
MEM



ACLK: AXI4 Clock

ARESETN: AXI4 Reset Not


ACLK
ARESETN
CPU
Read Request
Read Response
MEM
color fail


ACLK
ARESETN
ARADDR: Read Address
ARVALID: Read Address Valid
ARREADY: Read Address Accept Ready
CPU
MEM
AR: Address Read

AXI4 Handshaking

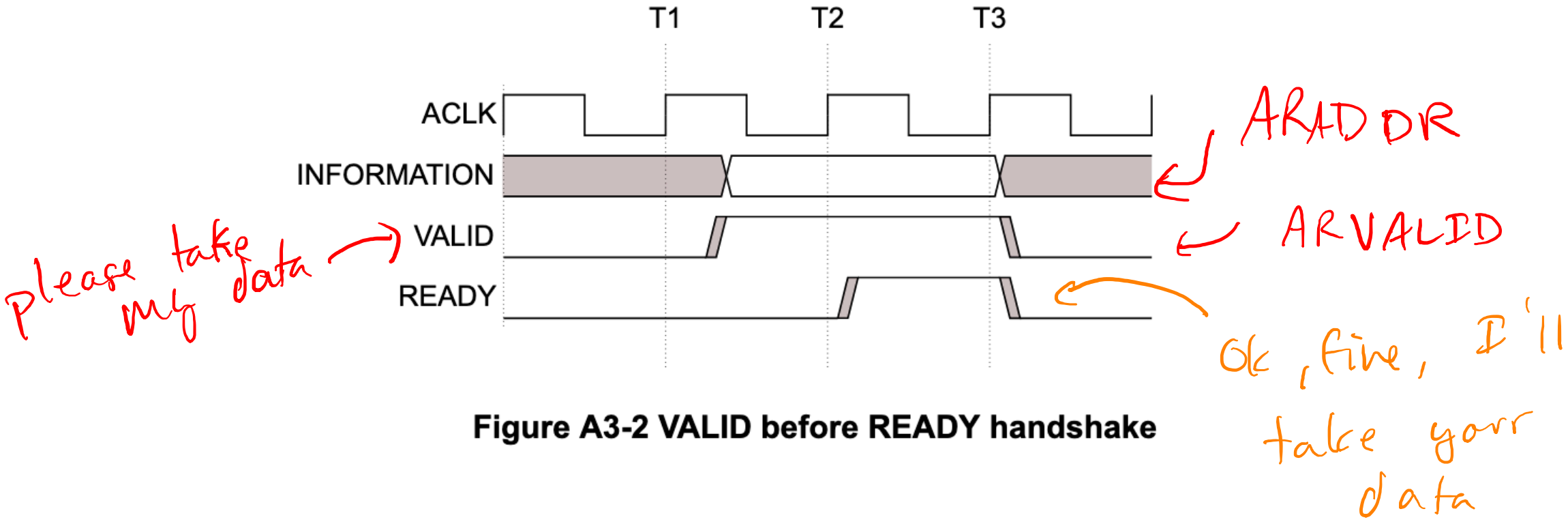
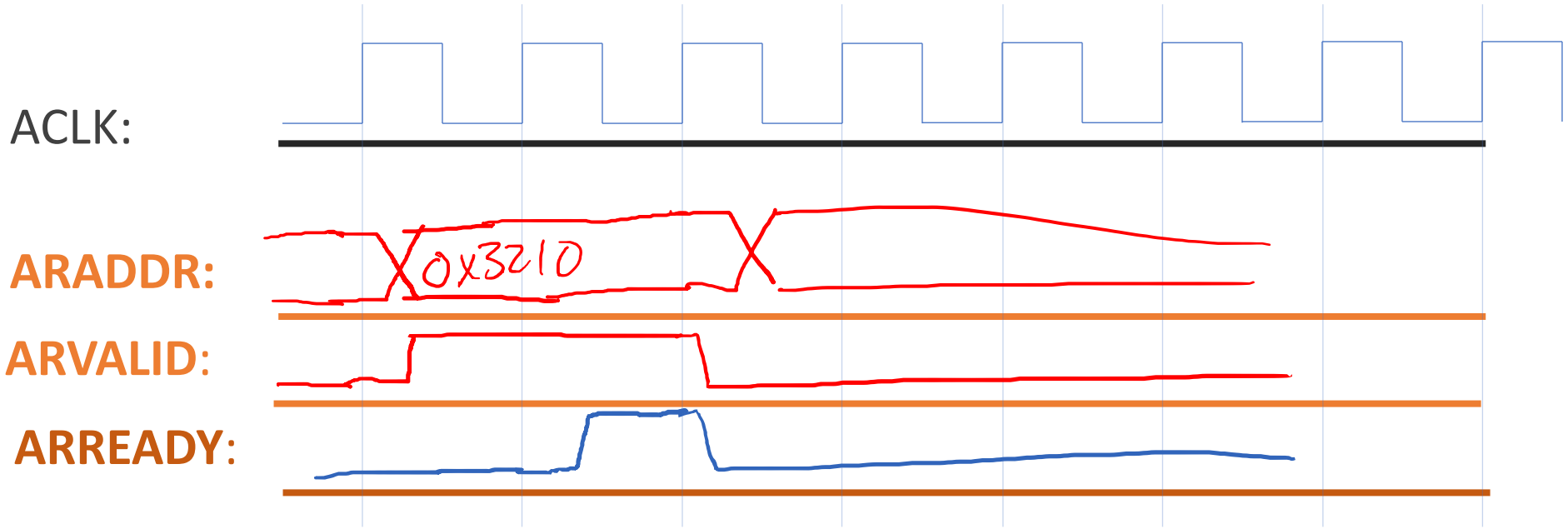


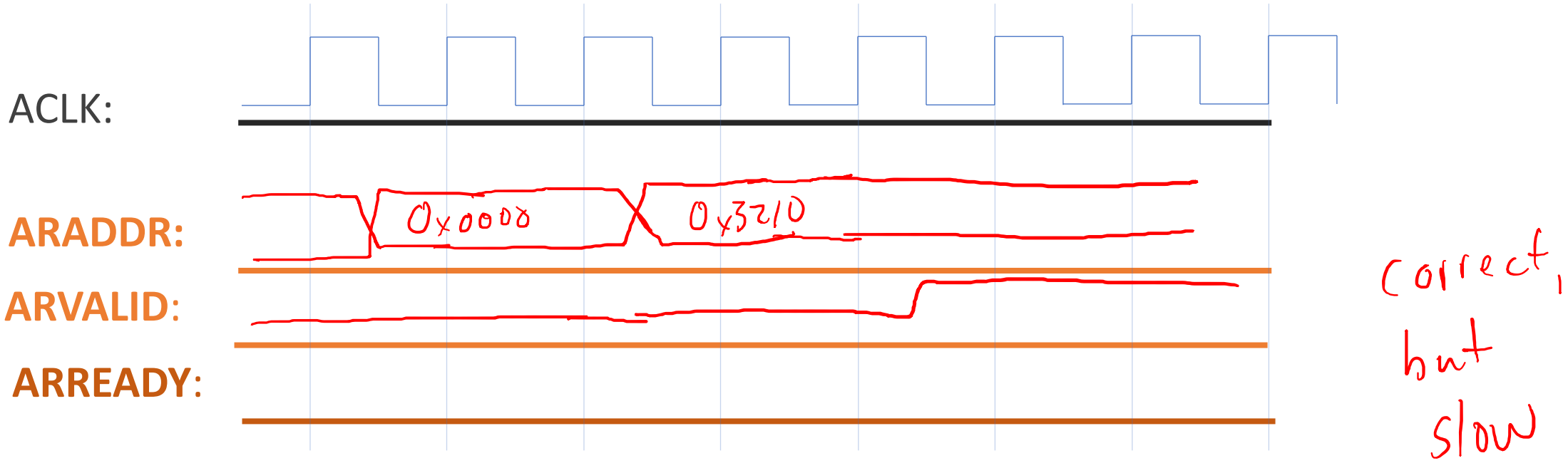
Figure A3-2 VALID before READY handshake

AXI4 Lite Read Transaction

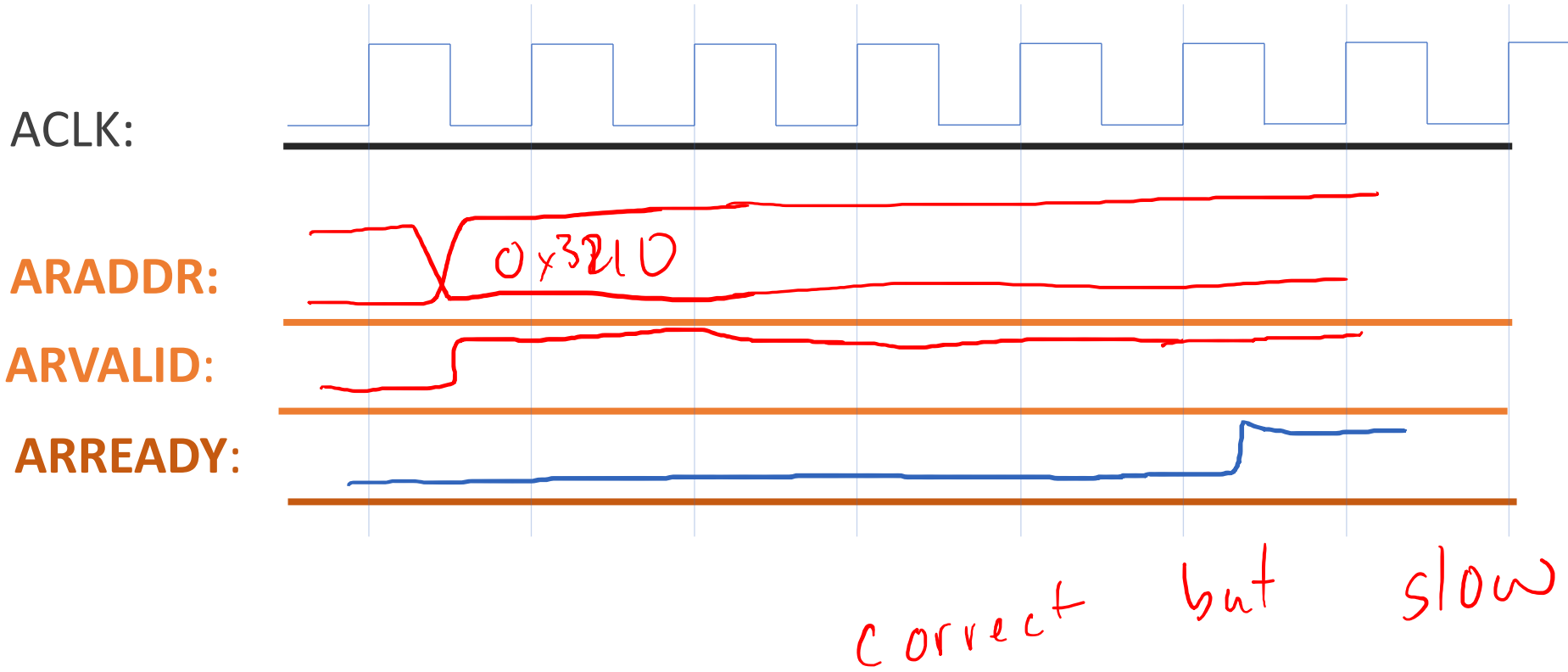


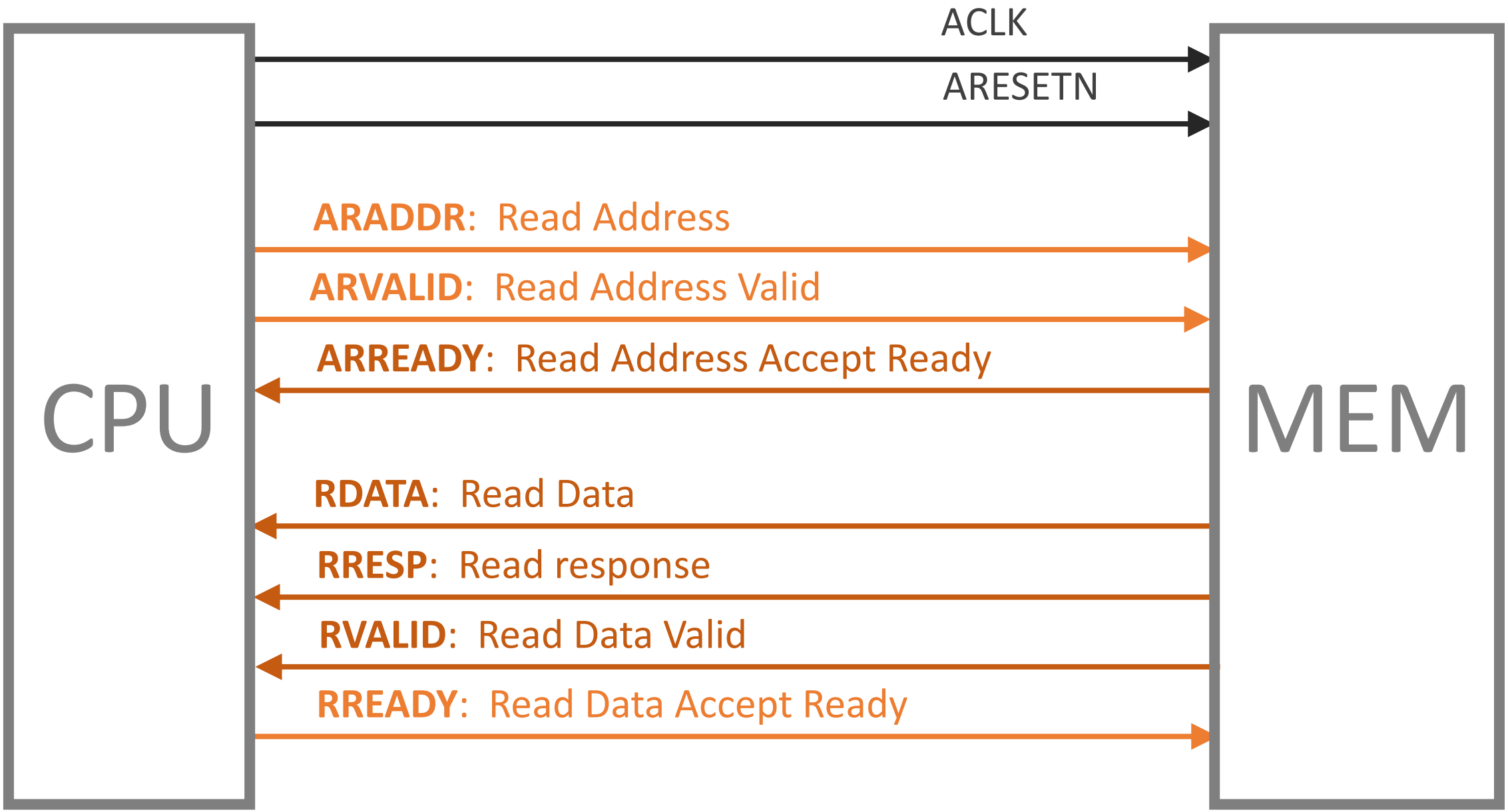
Addr: 0x3210

What if?


ACLK:
ARADDR:
0x0008
0x3210
ARVALID:
ARREADY:
correct, but slow

What if?




ACLK
ARESETN
ARADDR: Read Address
ARVALID: Read Address Valid
ARREADY: Read Address Accept Ready
CPU
MEM
RDATA: Read Data
RRESP: Read response
RVALID: Read Data Valid
RREADY: Read Data Accept Ready

What is RRESP?

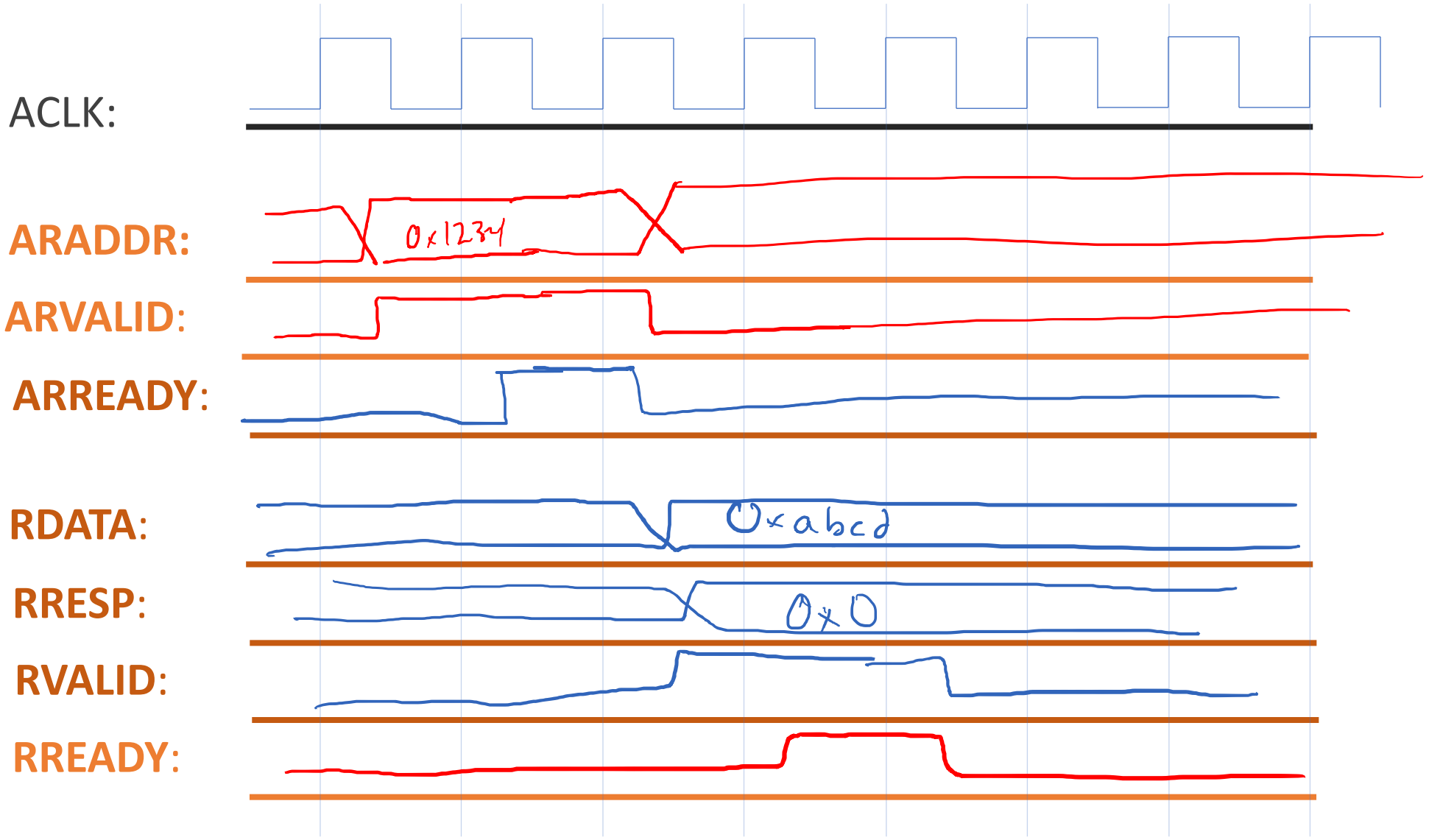
Table A3-4 RRESP and BRESP encoding

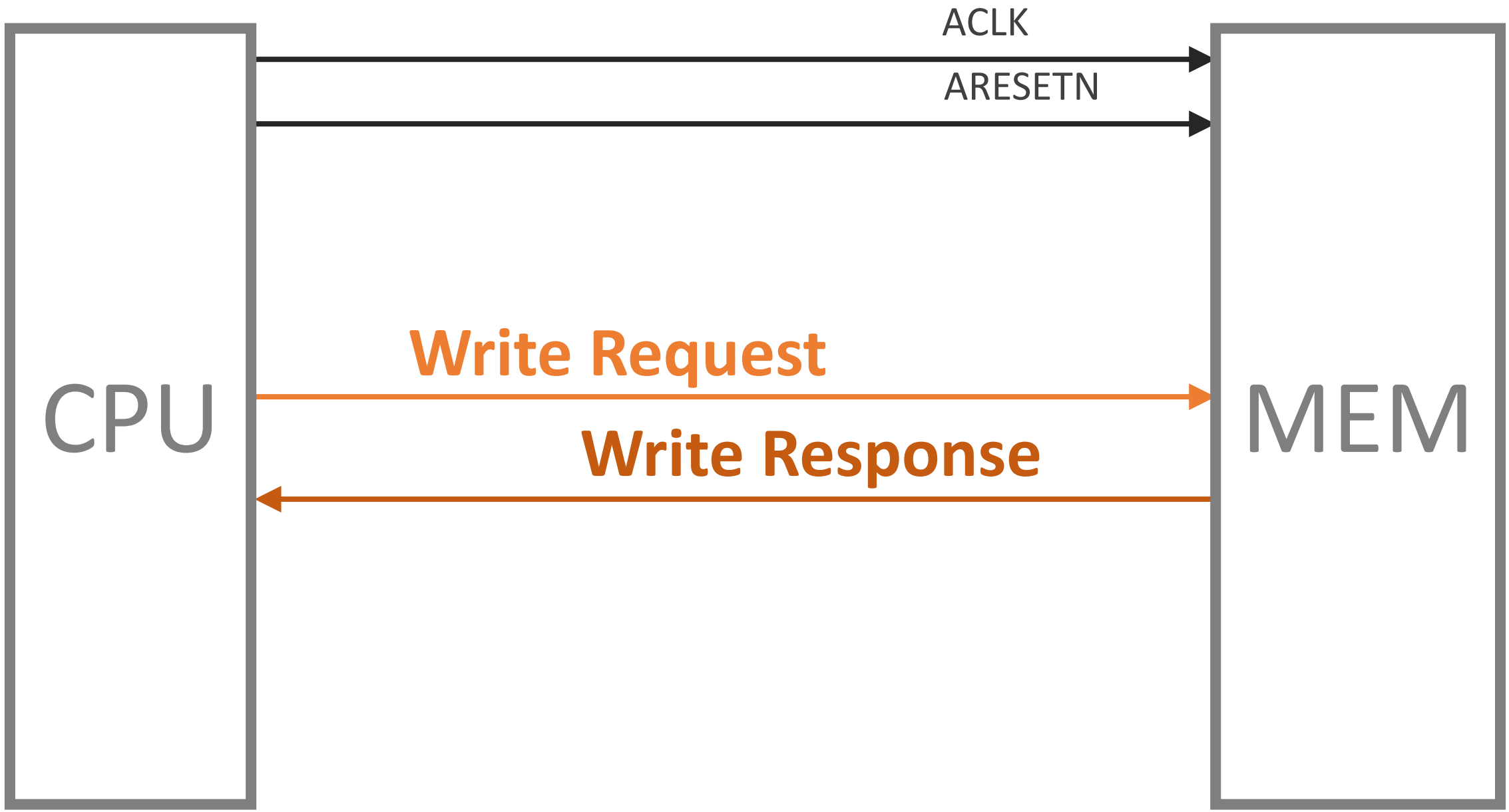
RRESP[1:0] BRESP[1:0]	Response
0b00	OKAY
0b01	EXOKAY
0b10	SLVERR
0b11	DECERR

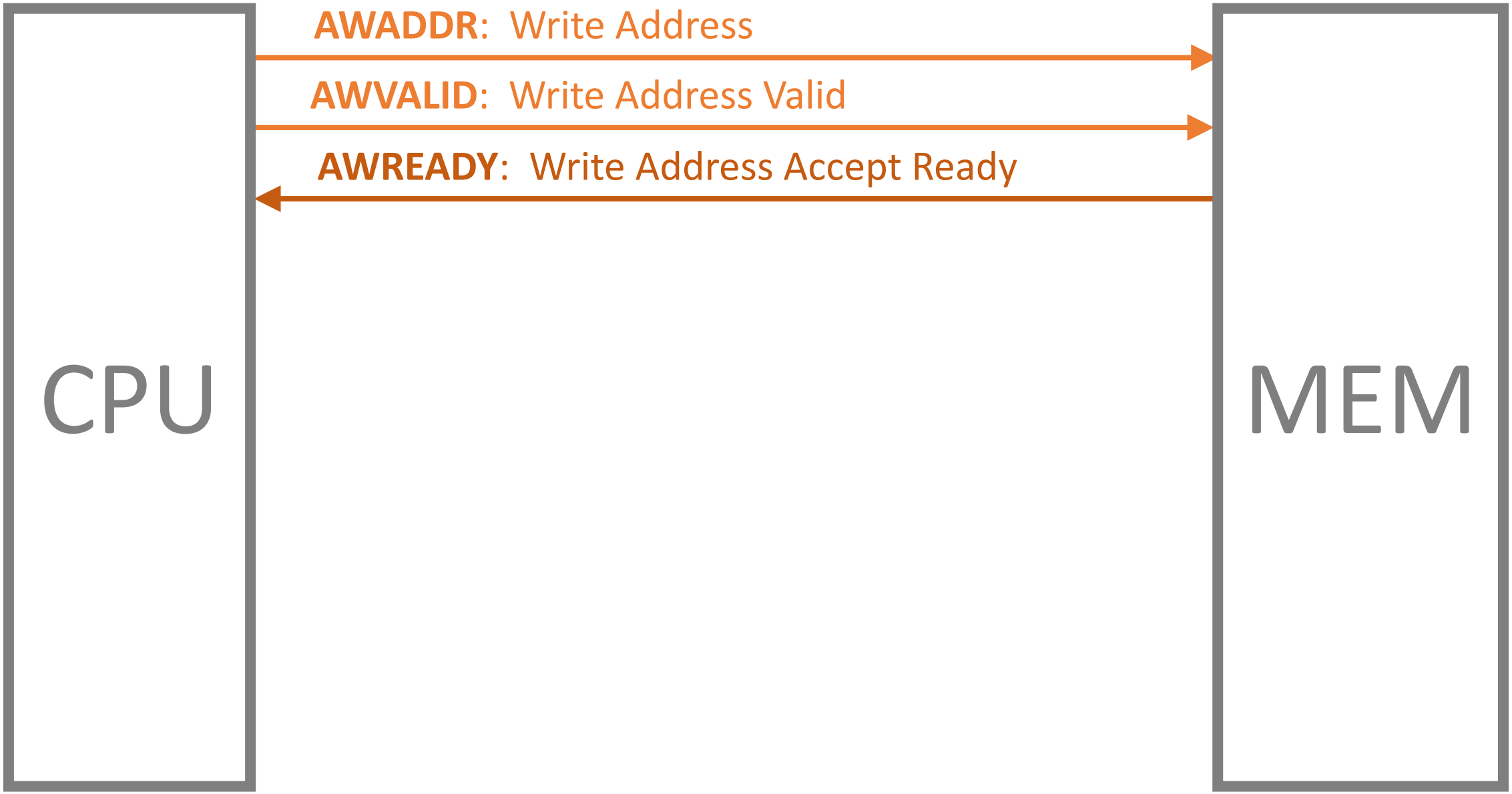
- Mostly used to send error codes back to CPU
- We'll always just use 0b00

Load 0x1234, response: 0xabcd

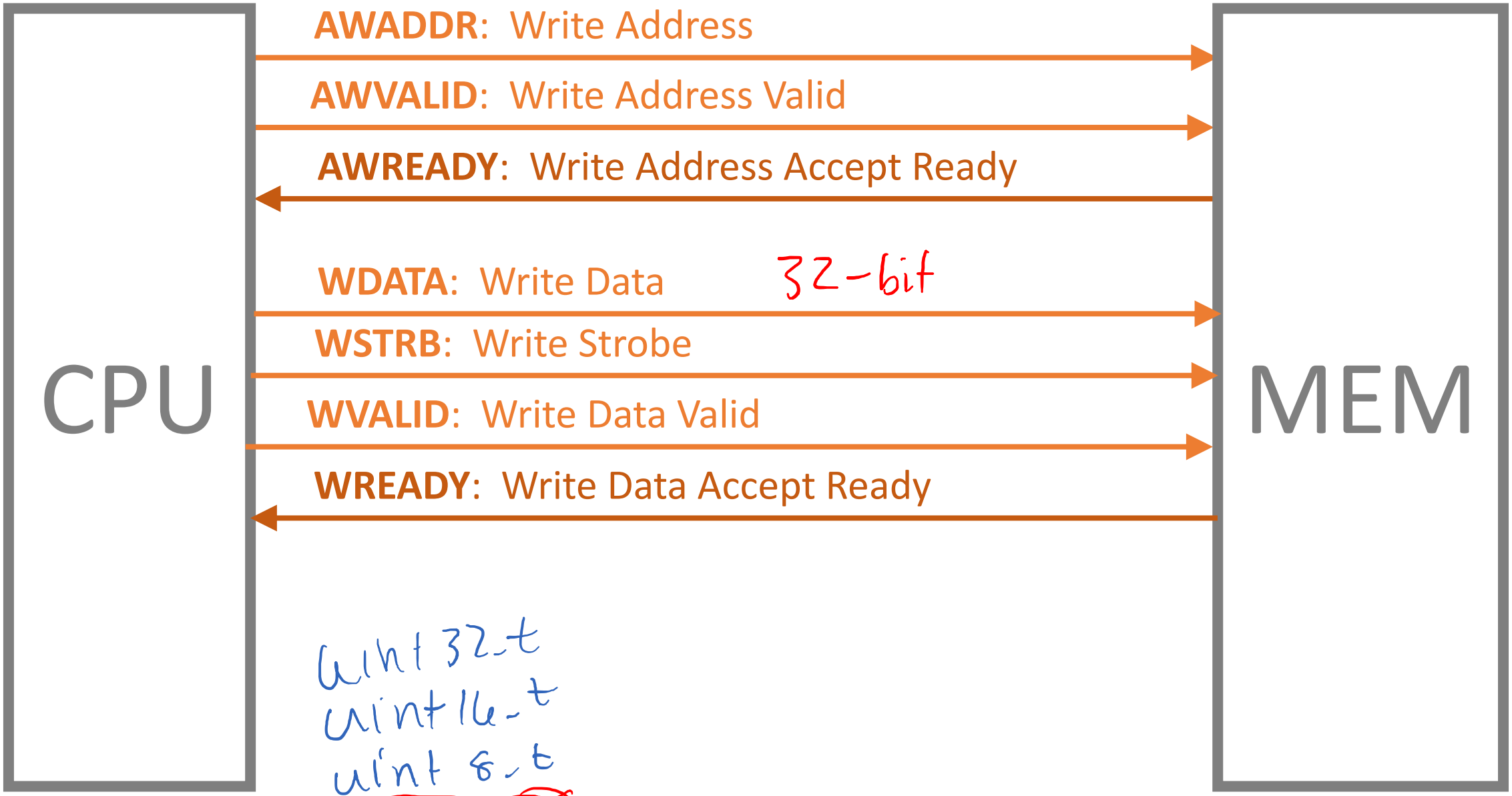
assume ARESETN = 1




ACLK
ARESETN
Write Request
Write Response
CPU
MEM



ACLK and ARESETN not shown



uint32_t
uint16_t
uint8_t
~~uint4_t~~

ACLK and ARESETN not shown

Q: How do you send a 1-byte (8-bit) value on a 32-bit bus?

- A: WSTB: Write Strobe

What is WSTRB?

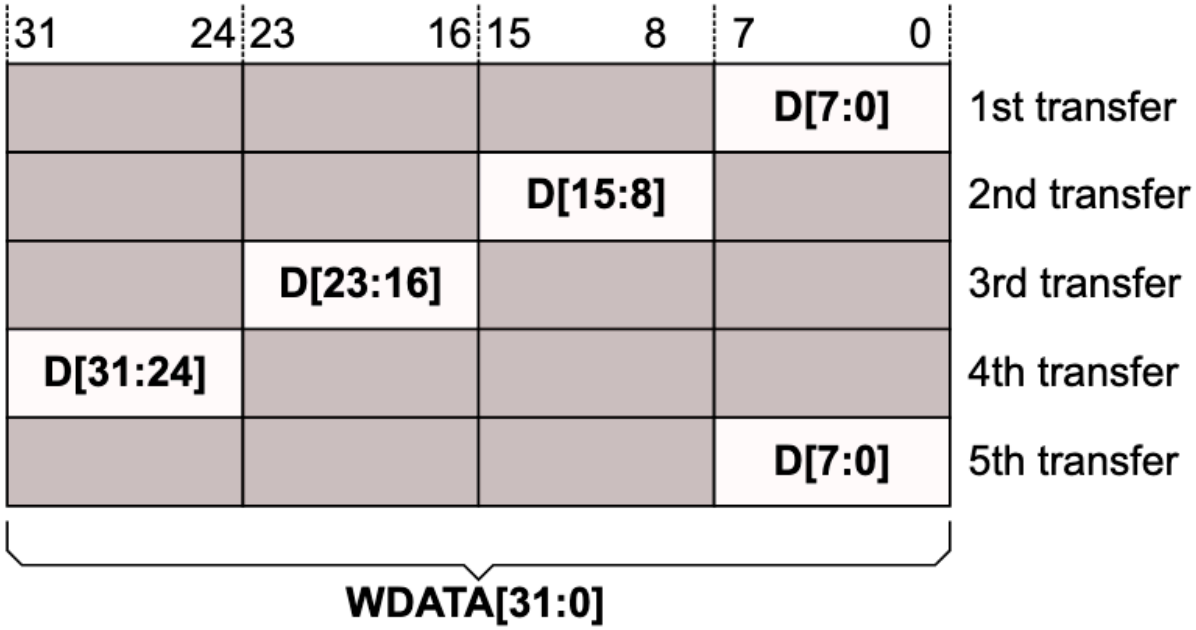
The **WSTRB[n:0]** signals when **HIGH**, specify the byte lanes of the data bus that contain valid information. There is one write strobe for each eight bits of the write data bus, therefore **WSTRB[n]** corresponds to **WDATA[(8n)+7: (8n)]**

What is WSTRB here?

| D[15:8] | D[7:0]

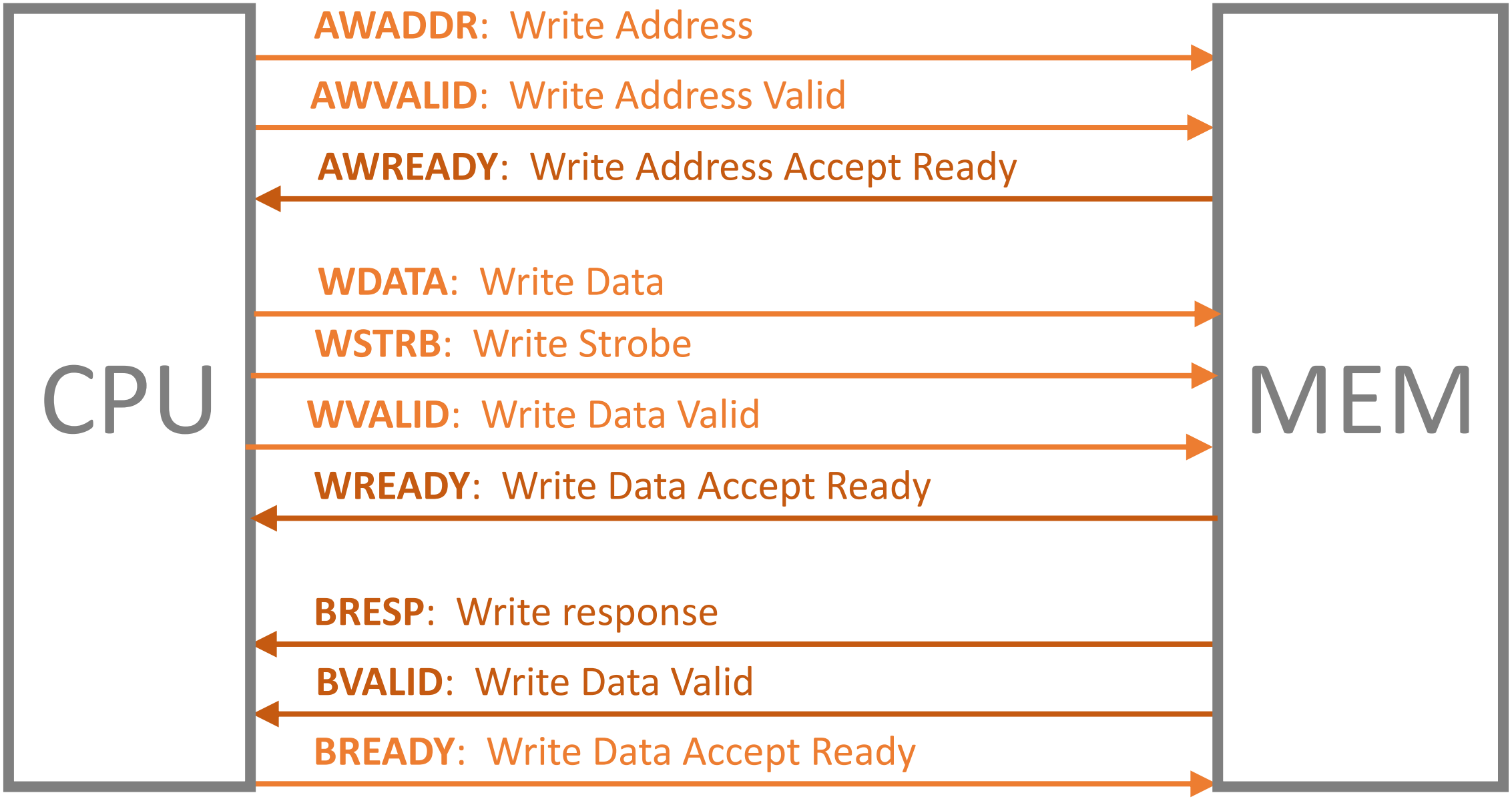
0b0011

WSTRB:



0b0001
0b0010
0b0100
0b1000
0b0001

Figure A3-8 Narrow transfer example with 8-bit transfers



ACLK and ARESETN not shown

BRESP is just like RRESP

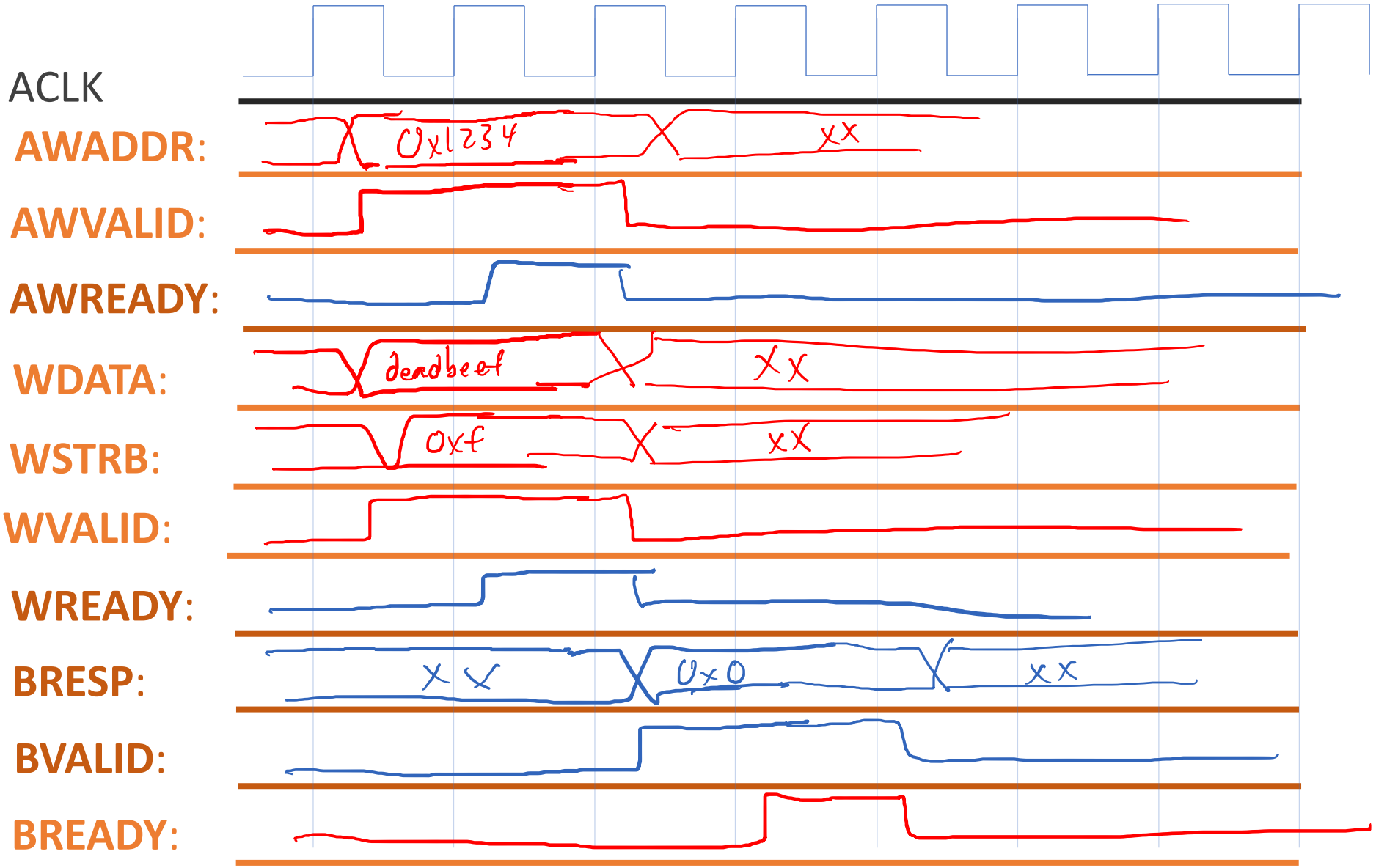
Table A3-4 RRESP and BRESP encoding

RRESP[1:0] BRESP[1:0]	Response
0b00	OKAY
0b01	EXOKAY
0b10	SLVERR
0b11	DECERR

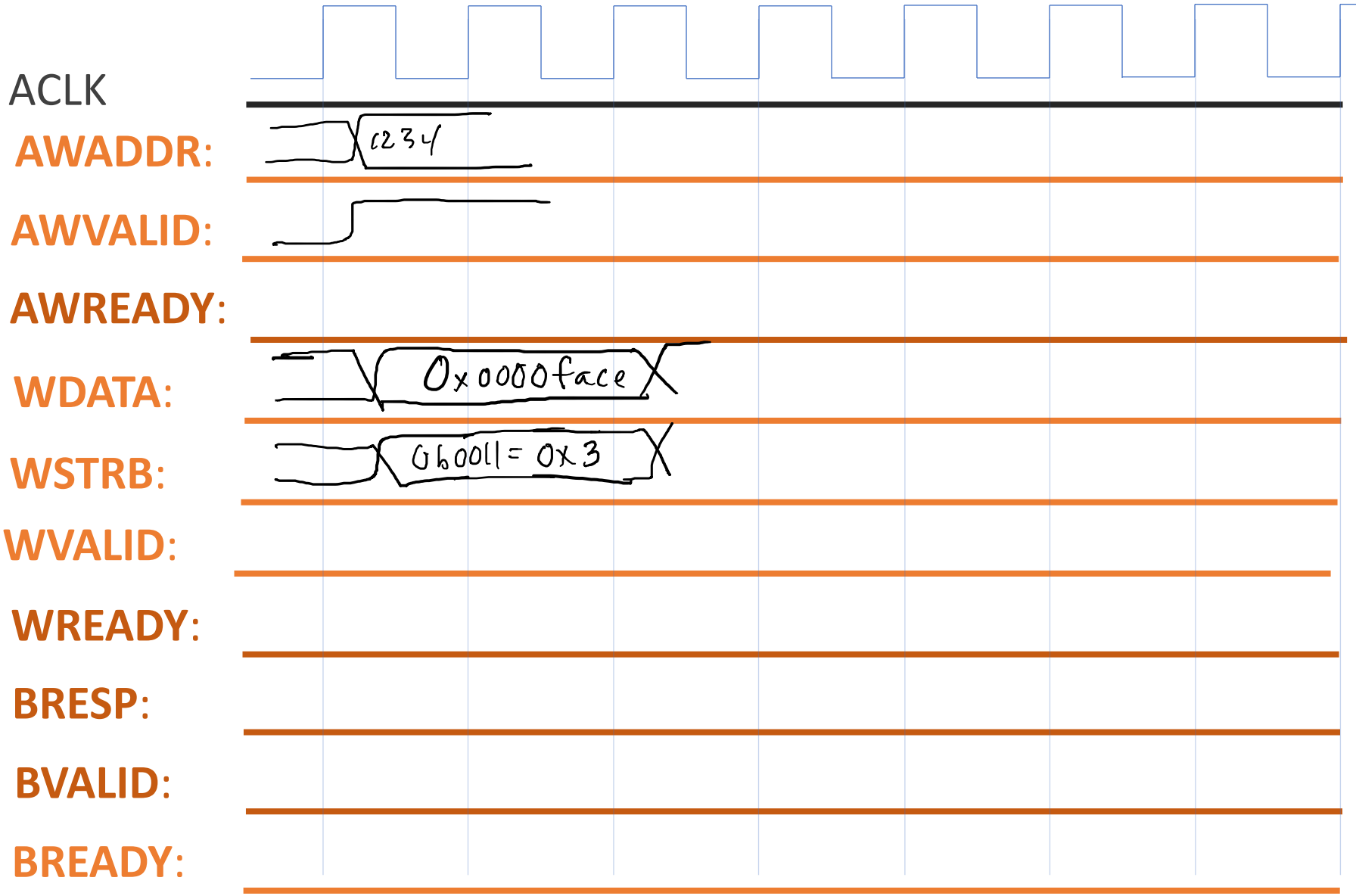
- Mostly used to send error codes back to CPU
- We'll always just use 0b00

Writing 0xdeadbeef to 0x1234

0b 1111 = 0xf



Writing 0xface to 0x1234



What about 0xface to 0x1236

0x1234 + 2

⇒ WDATA = 0xface0000

WSTRB = 0b1100 = 12 = 0xC

Next Time

- High-Performance Busses

References

- https://www.youtube.com/watch?v=okiTzvihHRA
- https://web.eecs.umich.edu/~prabal/teaching/eecs373/
- https://en.wikipedia.org/wiki/File:Computer_system_bus.svg
- https://www.realdigital.org/doc/a9fee931f7a172423e1ba73f66ca4081

- AMBA® AXI™ and ACE™ ProtocolSpecification

02: AXI4

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University